US007428535B1

(12) **United States Patent**
Peyton et al.

(10) **Patent No.: US 7,428,535 B1**
(45) **Date of Patent: Sep. 23, 2008**

(54) **AUTOMATIC RELEVANCE FILTERING**

(75) Inventors: **Eric S. Peyton**, Lisle, IL (US); **Tim W. Bumgarner**, Sharpsburg, MD (US); **Todd R. Fernandez**, Mountain View, CA (US); **David H. Soghoian**, Earlysville, VA (US)

(73) Assignee: **Apple Inc.**, Cupertino, CA (US)

(*) Notice: Subject to any disclaimer, the term of this patent is extended or adjusted under 35 U.S.C. 154(b) by 382 days.

(21) Appl. No.: **10/877,306**

(22) Filed: **Jun. 25, 2004**

(51) **Int. Cl.**
G06F 17/30 (2006.01)

(52) **U.S. Cl.** **707/5**; 707/102; 707/101

(58) **Field of Classification Search** 707/1–10, 707/100–104.1, 200–206, 5, 6, 101, 102; 705/26, 28; 715/501.1, 530, 115, 239, 728; 717/105

See application file for complete search history.

(56) **References Cited**

U.S. PATENT DOCUMENTS

5,729,730 A * 3/1998 Wlaschin et al. 707/3
6,449,627 B1 * 9/2002 Baer et al. 715/206
6,493,731 B1 * 12/2002 Jones et al. 715/501.1
6,505,219 B1 * 1/2003 MacLean et al. 715/530
7,346,844 B1 * 3/2008 Baer et al. 434/362
2002/0138358 A1 * 9/2002 Scheer 705/26
2002/0161674 A1 * 10/2002 Scheer 705/28
2006/0236304 A1 * 10/2006 Luo et al. 717/105

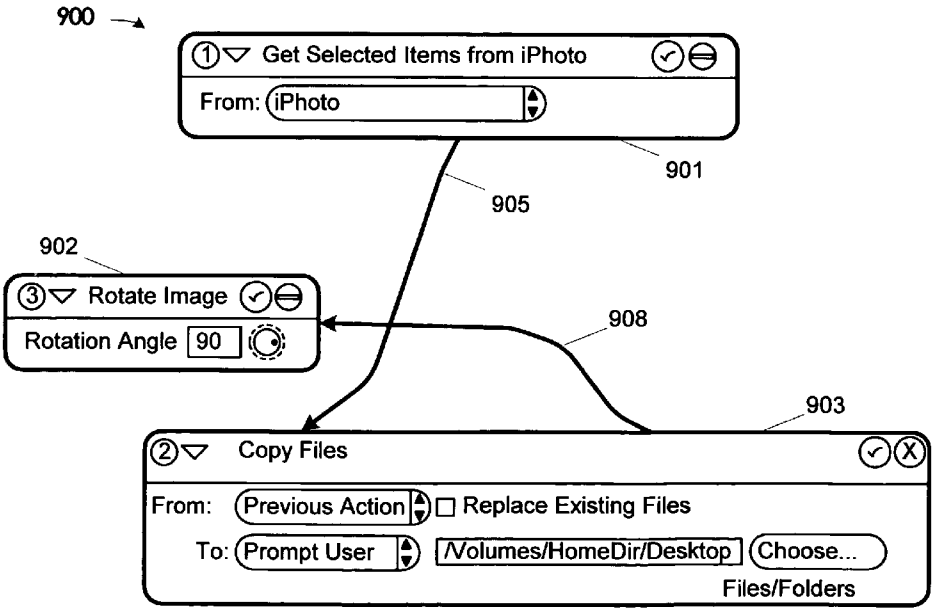
* cited by examiner

Primary Examiner—Diane Mizrahi
(74) *Attorney, Agent, or Firm*—William, Morgan & Amerson, P.C.

(57) **ABSTRACT**

A technique for use in a computer is disclosed. The method includes determining a context and a data requirement for a candidate action to be selected, the selection specifying an action in a workflow; and filtering the candidate actions for relevance in light of the context and the data requirement. The filtering includes one of according a relevance to each of the candidate actions relative to the degree to which data type conversion will be required between the selected action and the respective candidate action; and according a relevance to each of the candidate actions that require no data conversion to accept the output of the previously selected action as input. The apparatus, in a first aspect, includes a program storage medium encoded with instructions that, when executed by a computing device, performs the method. In a second aspect, the apparatus includes a computing apparatus programmed to perform the method.

40 Claims, 11 Drawing Sheets



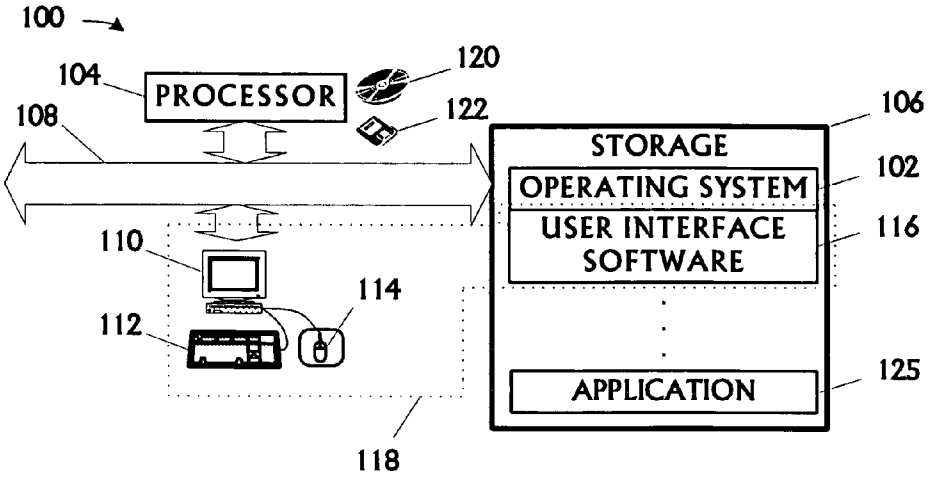

100
104
PROCESSOR
120
122
108
106
STORAGE
OPERATING SYSTEM
102
USER INTERFACE SOFTWARE
116
110
114
112
APPLICATION
125
118

FIG. 1

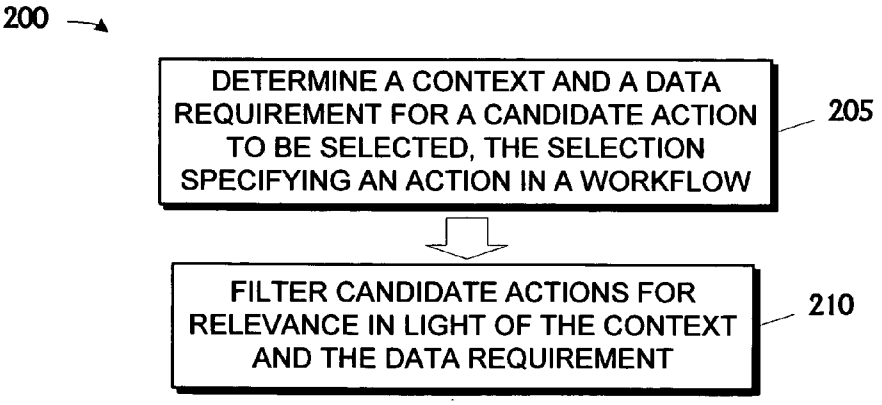

200
DETERMINE A CONTEXT AND A DATA REQUIREMENT FOR A CANDIDATE ACTION TO BE SELECTED, THE SELECTION SPECIFYING AN ACTION IN A WORKFLOW
205
FILTER CANDIDATE ACTIONS FOR RELEVANCE IN LIGHT OF THE CONTEXT AND THE DATA REQUIREMENT
210

FIG. 2

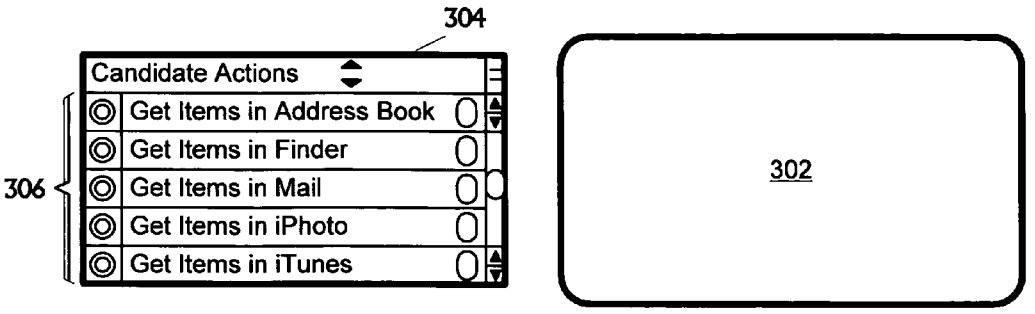

304
Candidate Actions
Get Items in Address Book
Get Items in Finder
Get Items in Mail
Get Items in iPhoto
Get Items in iTunes
306
302

FIG. 3A

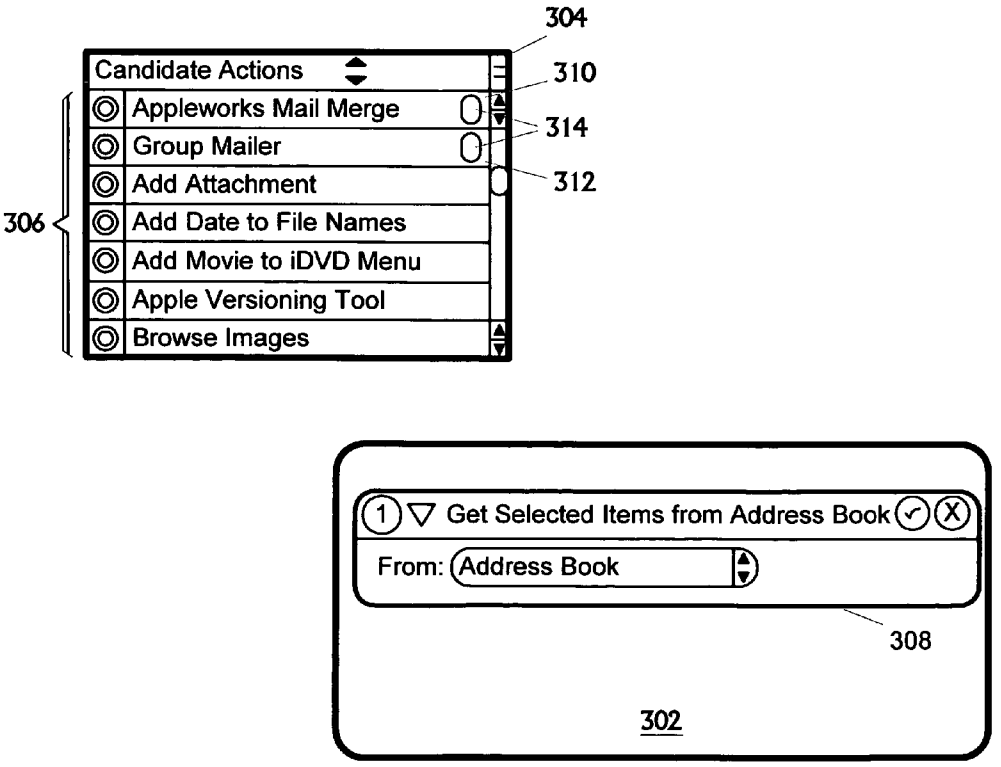
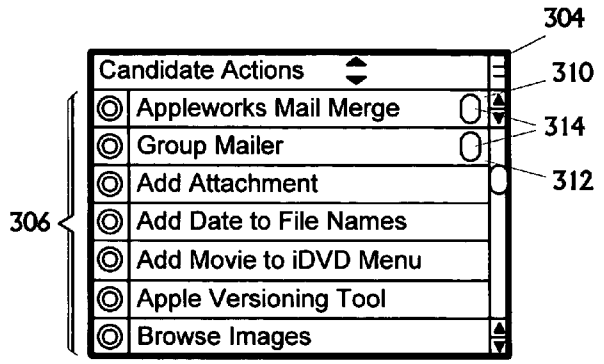

304
Candidate Actions
310
Appleworks Mail Merge
314
Group Mailer
312
Add Attachment
306
Add Date to File Names
Add Movie to iDVD Menu
Apple Versioning Tool
Browse Images
1
Get Selected Items from Address Book
From:
Address Book
308
302

FIG. 3B


304
Candidate Actions
310
Appleworks Mail Merge
314
Group Mailer
312
Add Attachment
306
Add Date to File Names
Add Movie to iDVD Menu
Apple Versioning Tool
Browse Images

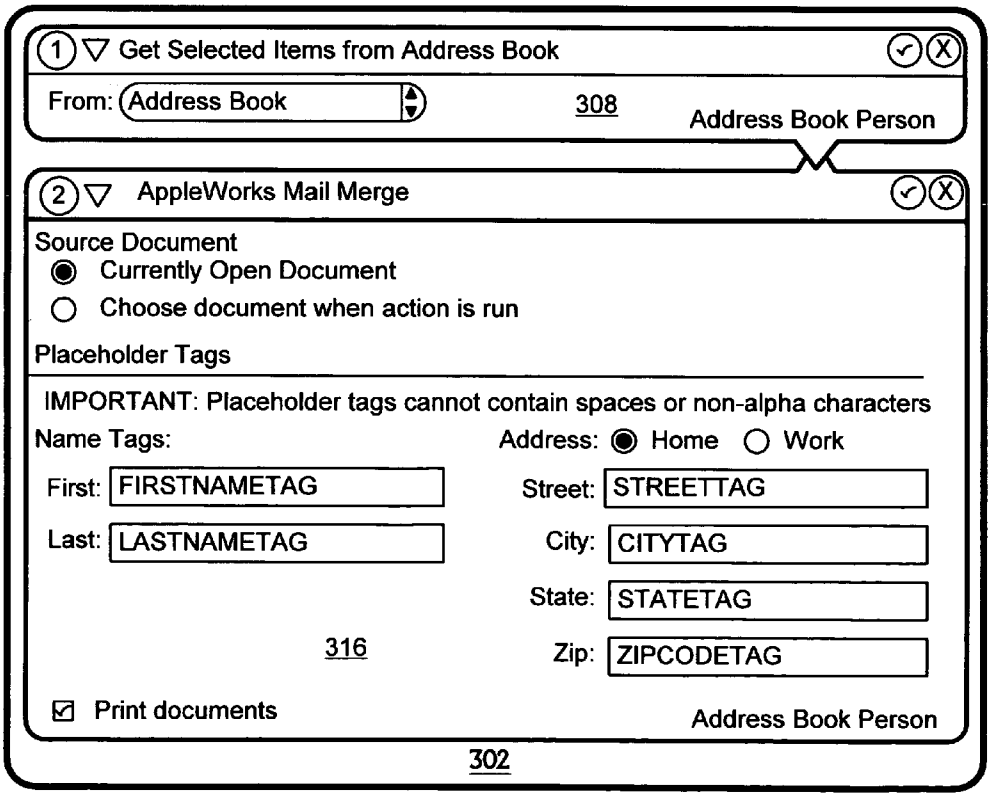

1 Get Selected Items from Address Book
From: Address Book
308
Address Book Person
2 AppleWorks Mail Merge
Source Document
Currently Open Document
Choose document when action is run
Placeholder Tags
IMPORTANT: Placeholder tags cannot contain spaces or non-alpha characters
Name Tags:
Address: Home Work
First: FIRSTNAMETAG
Street: STREETTAG
Last: LASTNAMETAG
City: CITYTAG
State: STATETAG
316
Zip: ZIPCODETAG
Print documents
Address Book Person
302

FIG. 3C

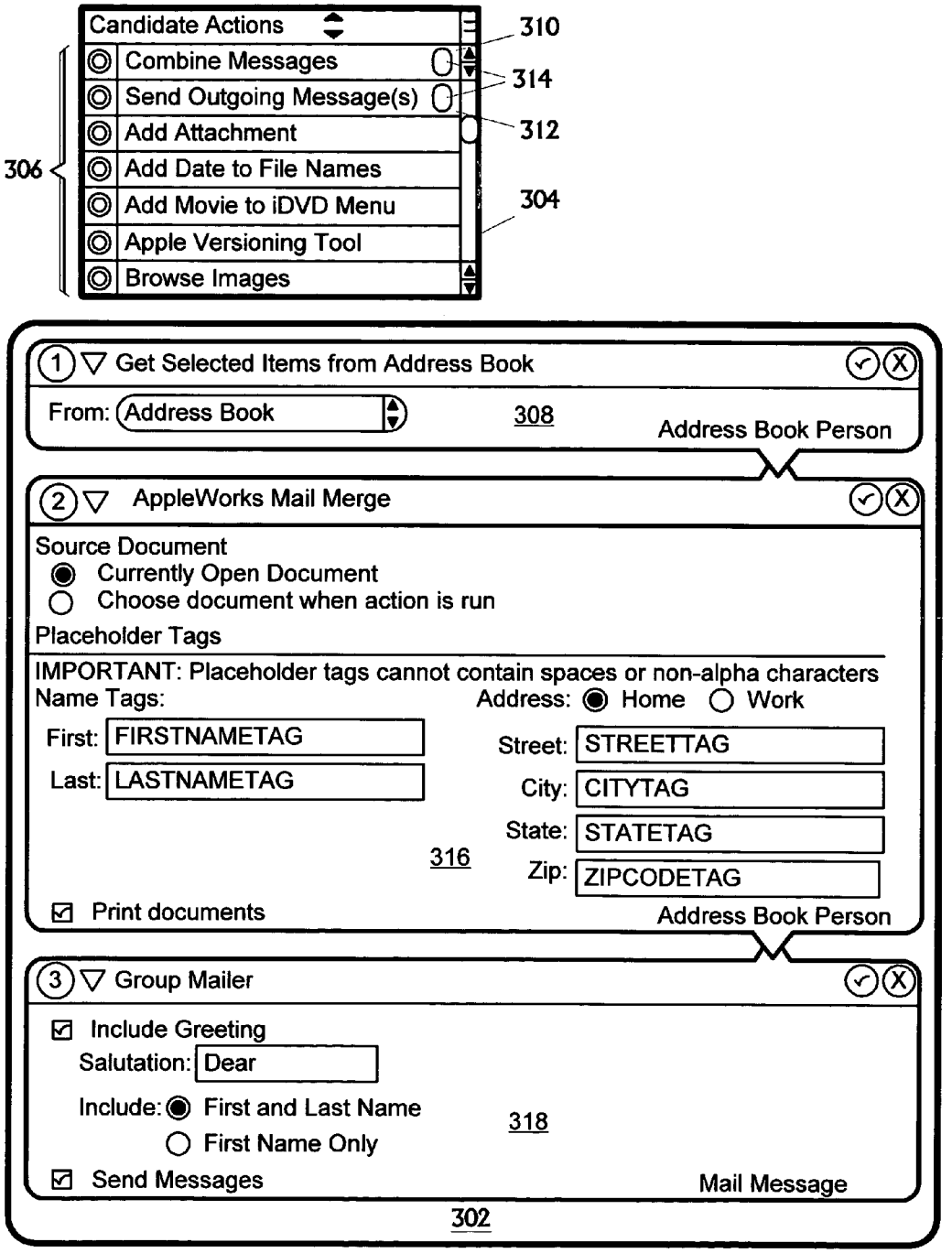

Candidate Actions
Combine Messages
Send Outgoing Message(s)
Add Attachment
Add Date to File Names
Add Movie to iDVD Menu
Apple Versioning Tool
Browse Images
310
314
312
304
306
1 Get Selected Items from Address Book
From: Address Book
308
Address Book Person
2 AppleWorks Mail Merge
Source Document
Currently Open Document
Choose document when action is run
Placeholder Tags
IMPORTANT: Placeholder tags cannot contain spaces or non-alpha characters
Name Tags:
First: FIRSTNAMETAG
Last: LASTNAMETAG
Address: Home Work
Street: STREETTAG
City: CITYTAG
State: STATETAG
Zip: ZIPCODETAG
316
Print documents
Address Book Person
3 Group Mailer
Include Greeting
Salutation: Dear
Include: First and Last Name
First Name Only
318
Send Messages
Mail Message
302

FIG. 3D

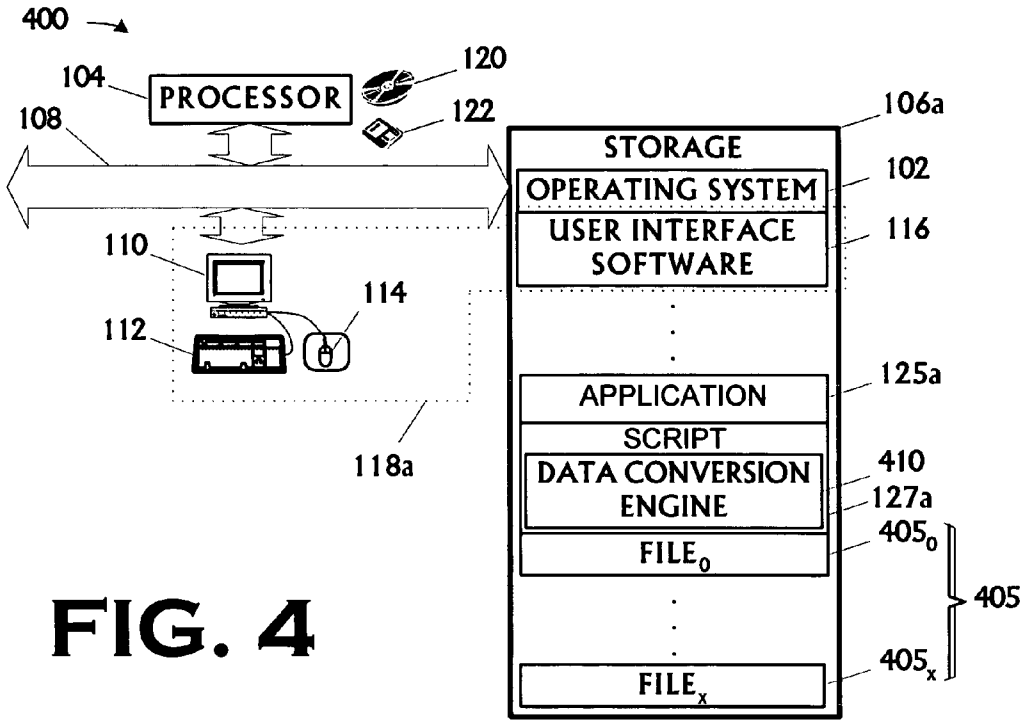

400
104
PROCESSOR
120
122
108
106a
STORAGE
102
OPERATING SYSTEM
USER INTERFACE SOFTWARE
116
110
114
112
125a
APPLICATION
SCRIPT
DATA CONVERSION ENGINE
410
127a
118a
FILE0
405
FILEx

FIG. 4

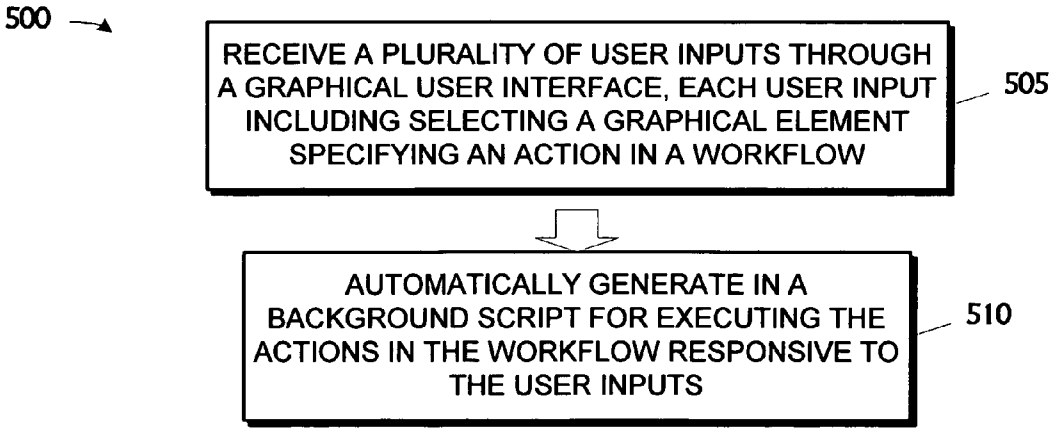

500
RECEIVE A PLURALITY OF USER INPUTS THROUGH A GRAPHICAL USER INTERFACE, EACH USER INPUT INCLUDING SELECTING A GRAPHICAL ELEMENT SPECIFYING AN ACTION IN A WORKFLOW
505
AUTOMATICALLY GENERATE IN A BACKGROUND SCRIPT FOR EXECUTING THE ACTIONS IN THE WORKFLOW RESPONSIVE TO THE USER INPUTS
510

FIG. 5

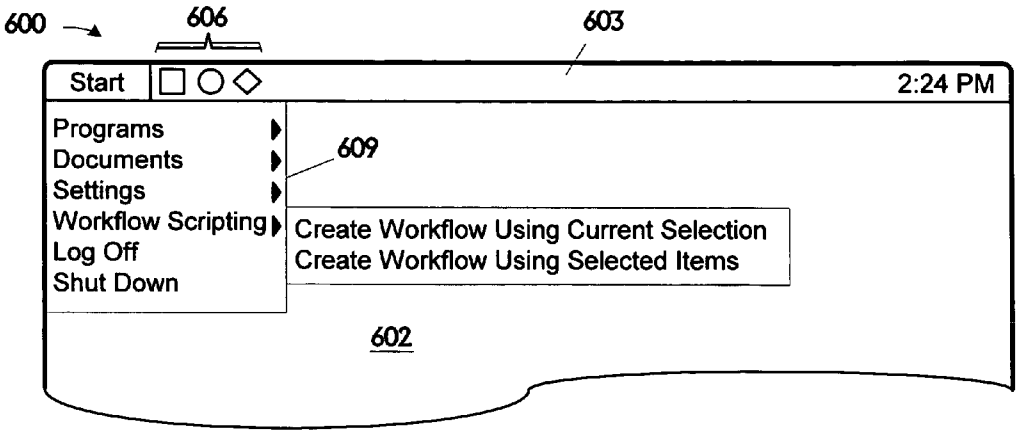

600
606
603
Start
2:24 PM
Programs
Documents
Settings
Workflow Scripting
Log Off
Shut Down
609
Create Workflow Using Current Selection
Create Workflow Using Selected Items
602

FIG. 6

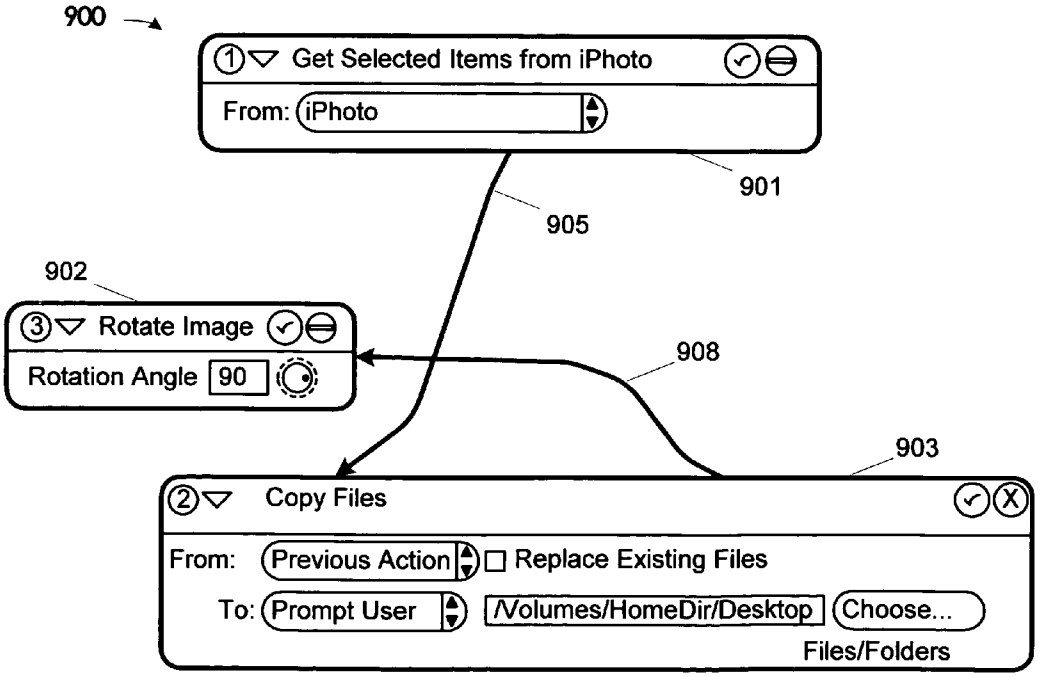

900
Get Selected Items from iPhoto
From: iPhoto
901
905
902
Rotate Image
Rotation Angle 90
908
903
Copy Files
From: Previous Action
Replace Existing Files
To: Prompt User
/Volumes/HomeDir/Desktop
Choose...
Files/Folders

FIG. 9

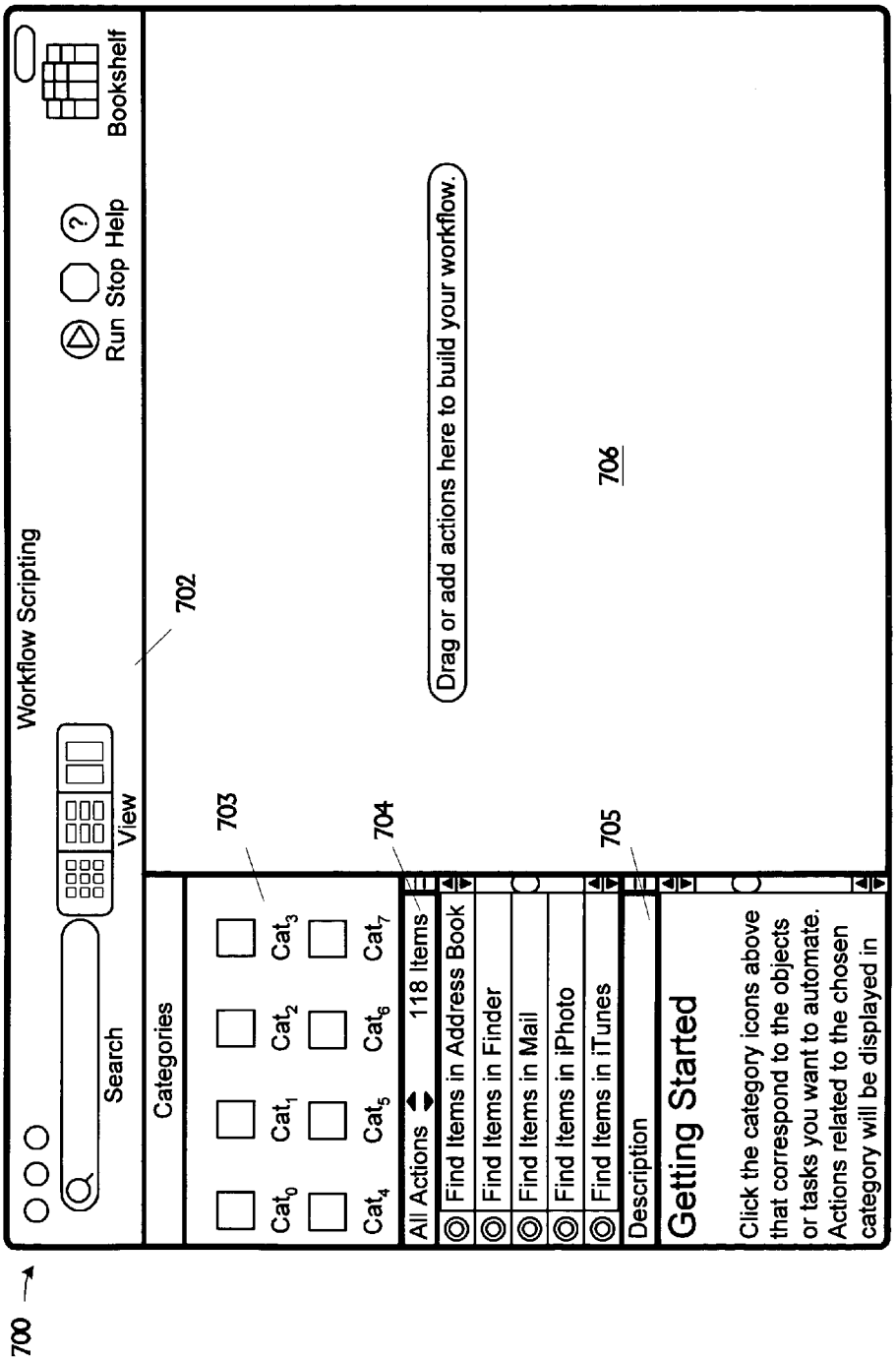

700
Workflow Scripting
Search
View
Run
Stop
Help
Bookshelf
702
Categories
Cat0
Cat1
Cat2
Cat3
Cat4
Cat5
Cat6
Cat7
703
704
All Actions
118 Items
Find Items in Address Book
Find Items in Finder
Find Items in Mail
Find Items in iPhoto
Find Items in iTunes
705
Description
Getting Started
Click the category icons above that correspond to the objects or tasks you want to automate. Actions related to the chosen category will be displayed in
Drag or add actions here to build your workflow.
706

FIG. 7

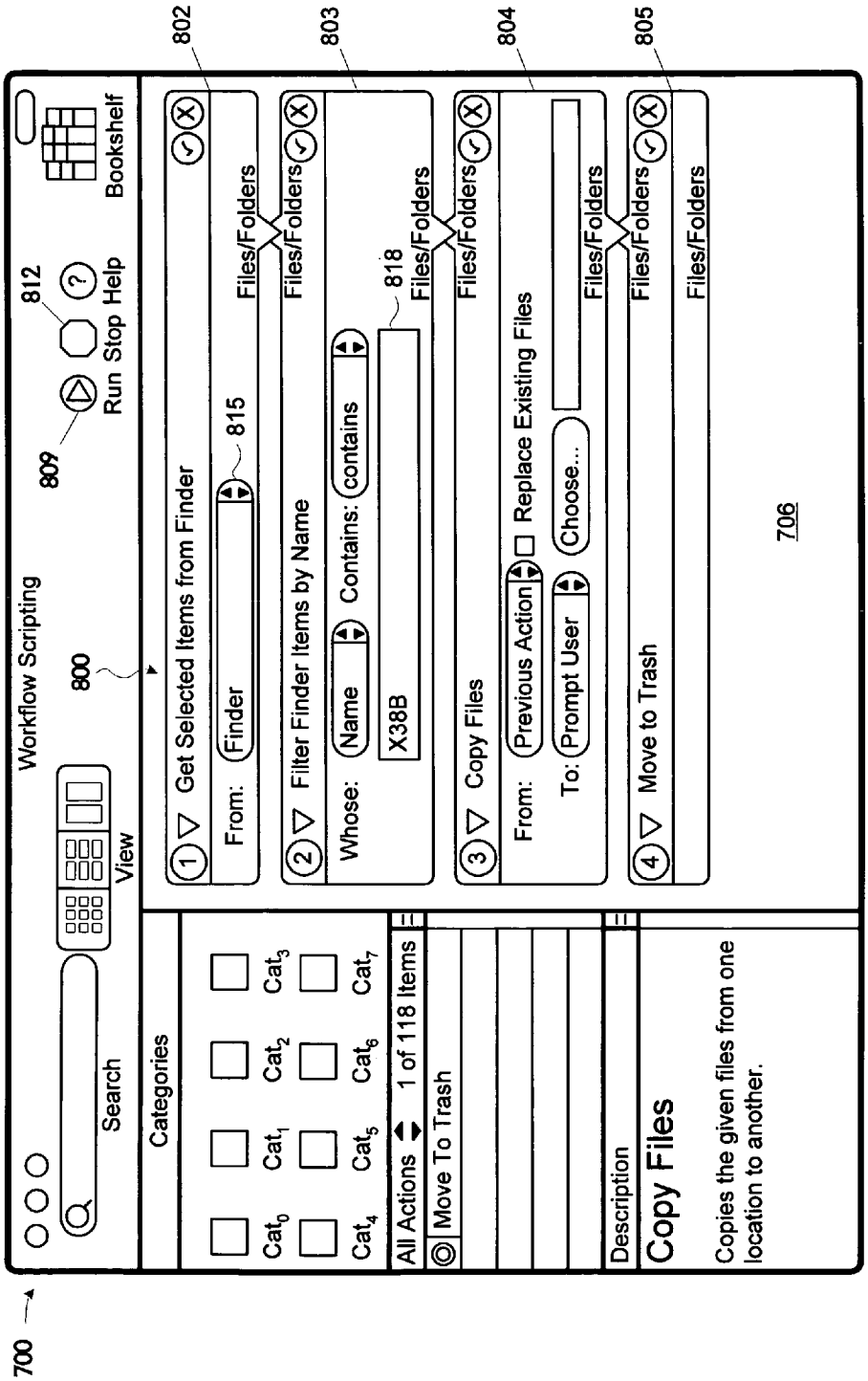

700
Workflow Scripting
800
Search
View
809
Run
812
Stop
Help
Bookshelf
Categories
Cat0
Cat1
Cat2
Cat3
Cat4
Cat5
Cat6
Cat7
All Actions
1 of 118 Items
Move To Trash
Description
Copy Files
Copies the given files from one location to another.
802
803
804
805
1
Get Selected Items from Finder
From:
Finder
815
Files/Folders
2
Filter Finder Items by Name
Whose:
Name
Contains:
contains
X38B
818
3
Copy Files
From:
Previous Action
Replace Existing Files
To:
Prompt User
Choose...
4
Move to Trash
706

FIG. 8

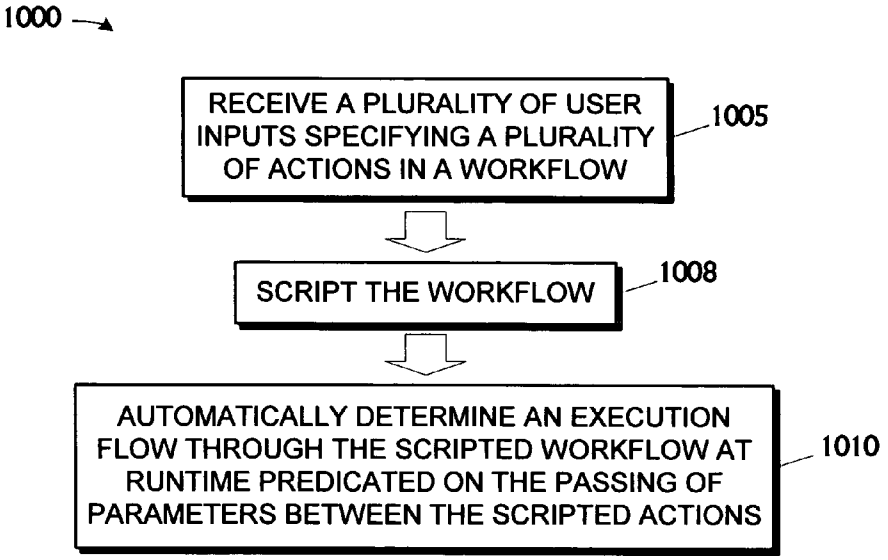

1000
RECEIVE A PLURALITY OF USER INPUTS SPECIFYING A PLURALITY OF ACTIONS IN A WORKFLOW
1005
SCRIPT THE WORKFLOW
1008
AUTOMATICALLY DETERMINE AN EXECUTION FLOW THROUGH THE SCRIPTED WORKFLOW AT RUNTIME PREDICATED ON THE PASSING OF PARAMETERS BETWEEN THE SCRIPTED ACTIONS
1010

FIG. 10

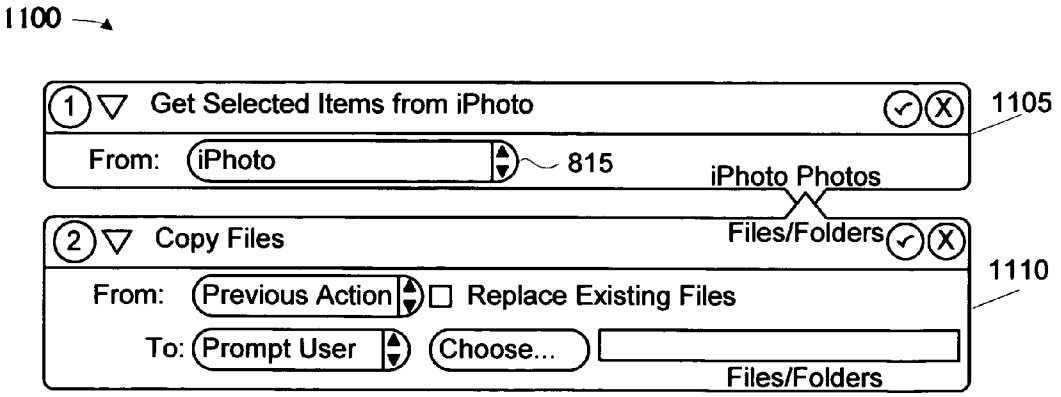

1100
1
Get Selected Items from iPhoto
1105
From:
iPhoto
815
iPhoto Photos
Files/Folders
2
Copy Files
1110
From:
Previous Action
Replace Existing Files
To:
Prompt User
Choose...
Files/Folders

FIG. 11

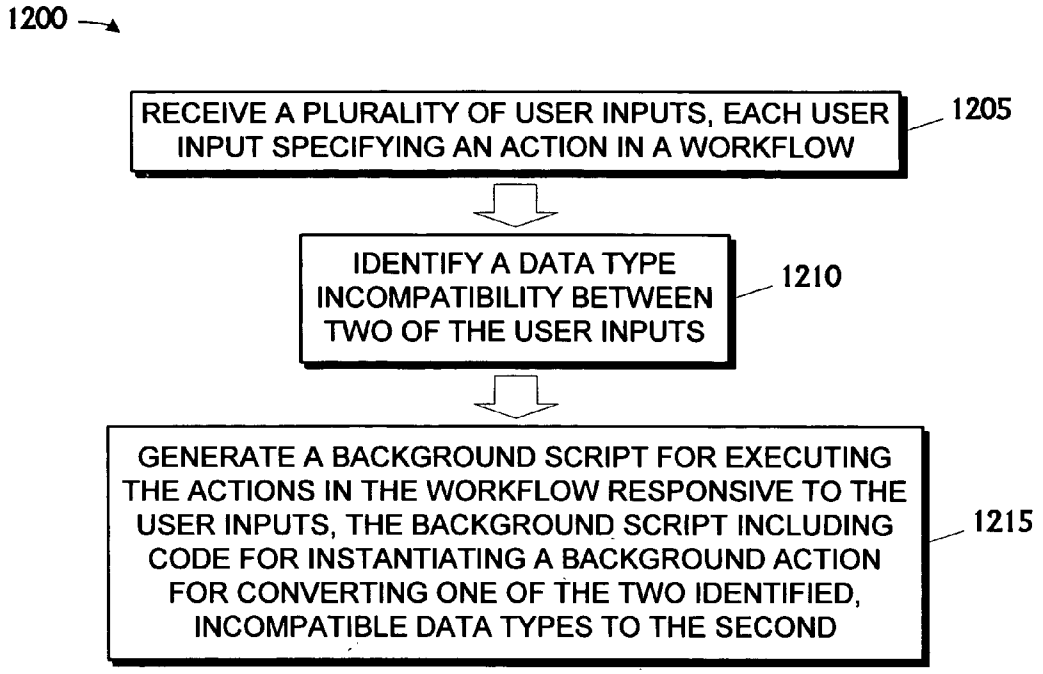


FIG. 12

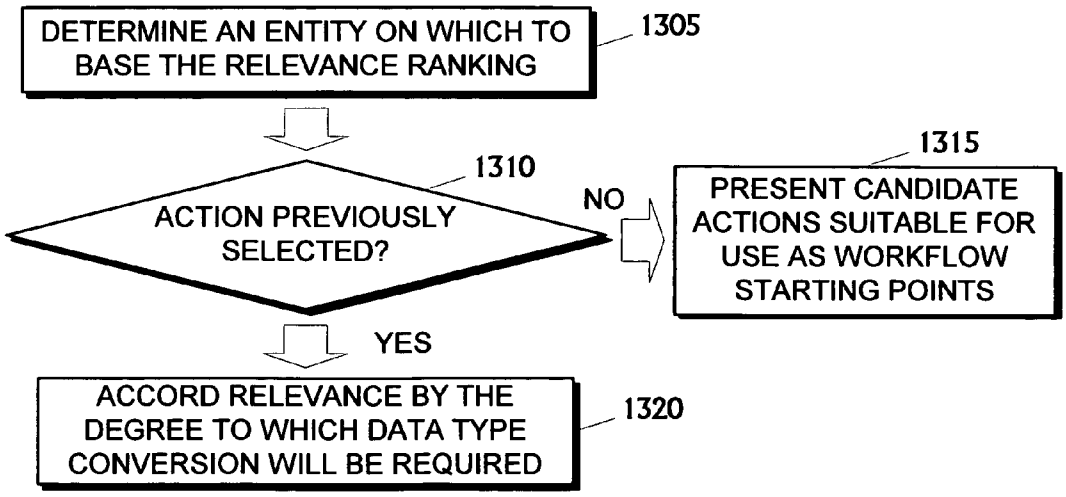


FIG. 13A

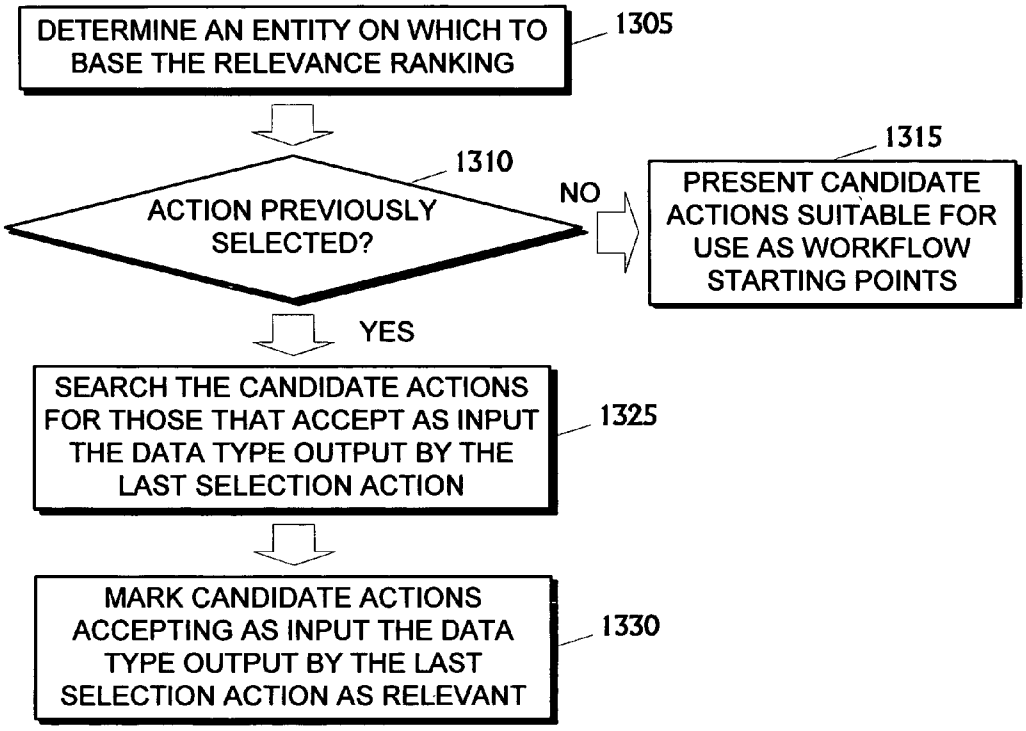

DETERMINE AN ENTITY ON WHICH TO BASE THE RELEVANCE RANKING
1305
ACTION PREVIOUSLY SELECTED?
1310
NO
PRESENT CANDIDATE ACTIONS SUITABLE FOR USE AS WORKFLOW STARTING POINTS
1315
YES
SEARCH THE CANDIDATE ACTIONS FOR THOSE THAT ACCEPT AS INPUT THE DATA TYPE OUTPUT BY THE LAST SELECTION ACTION
1325
MARK CANDIDATE ACTIONS ACCEPTING AS INPUT THE DATA TYPE OUTPUT BY THE LAST SELECTION ACTION AS RELEVANT
1330

FIG. 13B

AUTOMATIC RELEVANCE FILTERING

The present application is related to the following United States patent applications:

- U.S. patent application Ser. No. 10/877,292, Entitled, "Visual Programming Tool", filed in the names of Eric Peyton, et al. on an even date herewith; and commonly assigned herewith;
- U.S. patent application Ser. No. 10/876,940, Entitled, "Automatic Execution Flow Ordering", filed in the name of Eric Peyton on an even date herewith; and commonly assigned herewith;
- U.S. patent application Ser. No. 10/876,931, Entitled, "Automatic Conversion for Disparate Data Types", filed in the names of Eric Peyton and Tim Bumgarner on an even date herewith; and commonly assigned herewith.

Each of these applications is hereby incorporated by reference in full as if set forth herein verbatim.

BACKGROUND OF THE INVENTION

1. Field of the Invention

The present invention pertains to computer programming, and, more particularly, to a visual programming tool for a computing apparatus.

2. Description of the Related Art

Despite dramatic increases in recent years, some impediments to the use of computing technology by society at large still persist. Historically, the greatest impediment, aside from the computing capabilities themselves, was the user interface with the computer, or platform. At one time, the user interface consisted of a prompt at which a user entered a command. Typically, the command had to be selected from a list of commands whose content depended on the operating system under which the platform ran. Most operating systems provided some flexibility to the user by permitting the use of parameters, sometimes called "switches," in conjunction with the command. The user could then "program" the computer through the user interface to tailor the operation of the platform to their particular needs by entering the command and the appropriate parameters. The user could also open a text editor to create a text file that could be saved and used as a program.

This type of interface has some inherent drawbacks for programming. One significant drawback is that the commands and parameters must be memorized and/or looked up in a manual. This drawback is compounded by at least several factors. First, the commands and their parameters frequently are not intuitive, making them difficult to memorize. Second, the syntax for each command is typically very different. Third, the list of commands and parameters and the syntax frequently varied drastically depending on the type of operating system on which the platform was operating. These kinds of factors meant that the skills needed for interfacing with the computer were relatively specialized, and required some degree of time and effort to acquire. Consequently, the ability to work with computers remained limited to relatively small groups of people.

Much of this changed dramatically with the introduction of the "graphical user interface," or "GUI." The GUI takes the programming burden off the user by encapsulating the commands and their parameters in applications represented by graphical elements known as "icons" displayed to the user. The user then typically uses a pointing device, such as a mouse, a joystick, or a touchpad, to place a cursor over the icon. The choice is entered in manner depending on the type of pointing device employed. With a mouse, for instance, the user clicks a button, usually twice, to enter the selection. Thus, the user no longer needs to memorize lists of arcane commands and parameters, much less their syntax. Plus, the "drag and click" technique easily transfers to other platforms employing different GUIs. The user need only learn the functionality associated with the applications being selected.

The introduction of the GUI, coupled with dramatic increases in computing capabilities, opened the computing world to very many more users on a personal level. However, in taking the programming burden off the user, the encapsulation described above also inhibits the user's ability to program when they wish to do so. The GUI just does not include access to the programming levels of the operating system. The user can exit the GUI to enter a text-based interface such as the one first described. At that point, programming can occur. Or, the user can open a text editor with the GUI and enter text that will constitute a program. However, the user must still have previously acquired a working knowledge of the programming language, including commands, parameters, and syntax. But the GUI itself offers no way to visually program the platform without knowledge of a programming language's commands, parameters, and syntax.

Some attempts have been made to remedy this situation. For instance, an application known as "Khoros Cantata" uses graphical elements known as "glyphs" on a grid to replace lines of code to perform a kind of "visual programming." However, this technique only alleviates the need to manually enter the text of the program. The user is still required to program, and manual entry of values is still generally required. There is also a type of scripting where a user interface ("UI") is generated based on some criteria and then presented to the user at either design or runtime (e.g., Commando, Linux GUI builders, etc.). While the scripting may alleviate some repetition, it still nevertheless requires that the user, at least in the first instance, actually perform the programming.

One difficulty in visual programming of the type to which the present invention is pertinent is the ability to locate and identify suitable choices for incorporation into a scripted workflow. Today's computing platforms are typically robust and very capable. This means there are many potential choices, and not all of them will be relevant for inclusion. In some contexts, only a small percentage of the candidates for selection will be relevant in light of what has previously been scripted into the workflow. Thus, the process of identifying and locating relevant candidates for selection can be tedious and time-consuming.

Programs and utilities that support automated scripting exist to permit users to create scripts of user-level commands that may span disparate programs or contexts. These programs and utilities are often simple programs without extensible features or modular "programming" and/or without the ability to resolve incongruities in the "program" being scripted."

The present invention is directed to resolving, or at least reducing, one or all of the problems mentioned above.

SUMMARY OF THE INVENTION

The present invention, in its various aspects and embodiments, includes a computer-implemented method and an apparatus for use in a computing apparatus. The method comprises determining a context and a data requirement for a candidate action to be selected, the selection specifying an action in a workflow; and filtering the candidate actions for relevance in light of the context and the data requirement. The

apparatus, in a first aspect, comprises a program storage medium encoded with instructions that, when executed by a computing device, performs the method. In a second aspect, the apparatus comprises a computing apparatus programmed to perform the method. The method and apparatus are extensible and modular, to support flexible script creation, as needed by a user, and also involve features to resolve implicit incongruities in the work flow created by a user. Additional aspects of the invention include a framework and graphical user interface for creating user level work flows.

BRIEF DESCRIPTION OF THE DRAWINGS

The invention may be understood by reference to the following description in conjunction with the accompanying drawings, in which like reference numerals identify like elements, and in which:

FIG. **1** conceptually depicts, in a partial block diagram, selected portions of the hardware and software architectures, respectively, of a computing apparatus such as may be used to implement the present invention;

FIG. **2** illustrates one particular embodiment of a method for filtering candidate actions for presentation to a user during the visual programming of a workflow in accordance with the present invention;

FIG. **3**A-FIG. **3**D illustrate automatic relevance filtering in accordance with the present invention in one particular embodiment;

FIG. **4** conceptually illustrates, in a partial block diagram, selected portions of the hardware and software architectures, respectively, of one particular implementation of the computing apparatus of FIG. **1**;

FIG. **5** illustrates one particular embodiment for scripting a workflow in accordance with one particular embodiment of the present invention;

FIG. **6** depicts a portion of an exemplary screen display of the computing apparatus in FIG. **3**;

FIG. **7** depicts an empty window for workflow scripting in accordance with the present invention;

FIG. **8** depicts a defined and scripted workflow in the window of FIG. **7**;

FIG. **9** depicts several selected actions for use in illustrating a workflow ordering feature of some alternative embodiments of the present invention;

FIG. **10** illustrates one particular embodiment of a method in which the workflow through a plurality of selected actions is determined through the specified actions predicated on the passing of parameters between specified actions;

FIG. **11** depicts several selected actions for use in illustrating an automatic, seamless data type conversion feature of some alternative embodiments of the present invention;

FIG. **12** illustrates one particular embodiment of a method for automatic, seamless data type conversion; and

FIG. **13**A-FIG. **13**B illustrate two alternative techniques for determining the relevance of candidate actions.

While the invention is susceptible to various modifications and alternative forms, the drawings illustrate specific embodiments herein described in detail by way of example. It should be understood, however, that the description herein of specific embodiments is not intended to limit the invention to the particular forms disclosed, but on the contrary, the intention is to cover all modifications, equivalents, and alternatives falling within the spirit and scope of the invention as defined by the appended claims.

DETAILED DESCRIPTION OF THE INVENTION

Illustrative embodiments of the invention are described below. In the interest of clarity, not all features of an actual implementation are described in this specification. It will of course be appreciated that in the development of any such actual embodiment, numerous implementation-specific decisions must be made to achieve the developers' specific goals, such as compliance with system-related and business-related constraints, which will vary from one implementation to another. Moreover, it will be appreciated that such a development effort, even if complex and time-consuming, would be a routine undertaking for those of ordinary skill in the art having the benefit of this disclosure.

FIG. **1** depicts selected portions of the hardware and software architectures of a computing apparatus **100** programmed and operated in accordance with the present invention. Some aspects of the hardware and software architecture (e.g., the individual cards, the basic input/output system ("BIOS"), input/output drivers, etc.) are not shown. These aspects are omitted for the sake of clarity, and so as not to obscure the present invention. As will be appreciated by those of ordinary skill in the art having the benefit of this disclosure, however, the software and hardware architectures of the computing apparatus **100** will include many such routine features that are omitted from FIG. **1**.

Note that the present invention is largely computer-implemented with user interaction. Some portions of the detailed descriptions herein are consequently presented in terms of a software implemented process involving symbolic representations of operations on data bits within a memory in a computing apparatus. These descriptions and representations are the means used by those in the art to most effectively convey the substance of their work to others skilled in the art. The process and operation require physical manipulations of physical quantities. Usually, though not necessarily, these quantities take the form of electrical, magnetic, or optical signals capable of being stored, transferred, combined, compared, and otherwise manipulated. It has proven convenient at times, principally for reasons of common usage, to refer to these signals as bits, values, elements, symbols, characters, terms, numbers, or the like.

It should be borne in mind, however, that all of these and similar terms are to be associated with the appropriate physical quantities and are merely convenient labels applied to these quantities. Unless specifically stated or otherwise as may be apparent, throughout the present disclosure, these descriptions refer to the action and processes of an electronic device, that manipulates and transforms data represented as physical (electronic, magnetic, or optical) quantities within some electronic device's storage into other data similarly represented as physical quantities within the storage, or in transmission or display devices. Exemplary of the terms denoting such a description are, without limitation, the terms "processing," "computing," "calculating," "determining," "displaying," and the like.

Note also that the software implemented aspects of the invention are typically encoded on some form of program storage medium or implemented over some type of transmission medium. The program storage medium may be magnetic (e.g., a floppy disk or a hard drive) or optical (e.g., a compact disk read only memory, or "CD ROM"), and may be read only or random access. Similarly, the transmission medium may be twisted wire pairs, coaxial cable, optical fiber, or some other suitable transmission medium known to the art. The invention is not limited by these aspects of any given implementation.

Returning now to FIG. **1**, in the illustrated embodiment, the computing apparatus **100** is a Power Mac® personal computer available from Apple Computer, Inc., employing a MacOS operating system **102**. However, the invention is not so limited. The computing apparatus **100** may be implemented in virtually any type of electronic computing apparatus such as a general purpose computer, PDA, cell phone, or any other device for which user-level programming, or scripting, is desirable. The computing apparatus **100** may also employ other, alternative, operating systems (e.g., Windows™-based or disk operating system ("DOS")-based, Linux, PalmOS, Symbian, etc). The computing apparatus **100** may be, for instance, a workstation, employing a UNIX-based operating system. The invention is platform independent in the sense that it can be any type of platform employing any type of operating system.

The computing apparatus **100** also includes a computing device, i.e., the processor **104**, communicating with storage **106** over a bus system **108**. The computing apparatus **100** includes a monitor **110**, keyboard **112**, and a mouse **114**, which together, along with their associated user interface software **116** residing on the storage **106** comprise a user interface **118**. The user interface **118** in the illustrated embodiment is a graphical user interface ("GUI"), but this is not necessary to the practice of the invention. Practically any type of user interface known to the art may be employed.

The processor **104** may be any suitable processor known to the art. For instance, the processor may be a general purpose microprocessor or a digital signal processor ("DSP"). In the illustrated embodiment, the processor **104** is a Power PC microprocessor manufactured by International Business Machines Corp. ("IBM"), but the invention is not so limited. In alternative embodiments, the processor **104** may be an Athlon™ 64-bit processor commercially available from Advanced Micro Devices, Inc. ("AMD"), a 64-bit UltraSPARC™ or the 32-bit microSPARC™ from Sun Microsystems, any of the Itanium™, or Pentium™, or Alpha™-class processors from Intel Corporation. The processor **104** may also be implemented as a processor set, including a general purpose microprocessor and a math or graphics co-processor, for instance. However, the invention is not limited to a computing apparatus employing a processor. In alternative embodiments, the computing device may be, for instance, a controller.

The storage **106** typically includes a variety of memory types such as read only memory ("ROM"), a cache, a hard disk, and random access memory ("RAM"), none of which are individually shown. The storage **106** may also, in some embodiments, include removable storage such as an optical disk **120**, or a floppy electromagnetic disk **122**, or some other form, such as a magnetic tape (not shown) or a zip disk (not shown). Note that the memory types will be implemented using various technologies. For instance, the optical disk **120** is an optical medium, while the floppy electromagnetic disk **122** is magnetic. Still other technologies may be used to implement the various technology types, although most will be either magnetic or optical.

The bus system **108** similarly may employ any bus technology and operate in accordance with any suitable bus protocol known to the art. The illustrated embodiment, for instance, operates under the Peripheral Component Interconnect ("PCI") or NuBus protocols. However, numerous other bus protocols are well known and may be employed. Those in the art having the benefit of this disclosure will appreciate that the bus system **108** will typically comprise several electrically interconnected buses (not individually shown). For instance, a command bus, a data bus, and an expansion bus are frequently seen in suitable bus systems. The present invention nevertheless admits wide variation in implementing the bus system **108**, as well as the processor **104** and storage **106**. Thus, the invention is not limited by these aspects of any particular implementation of the computing apparatus **100**.

An application **125** also resides in the storage **106**. The application **125**, when invoked by a user through the user interface **118**, performs a method in accordance with the present invention with the interaction of the user. More particularly, the application **125** performs a method for visually programming a computing apparatus. One particular embodiment **200** of this method is illustrated in FIG. **2**. As was noted above, the present invention is platform independent in the sense that it can be practiced on practically any computing platform. However, for the sake of clarity, the method **200** in FIG. **2** shall be disclosed in the context of the computing apparatus **100** of FIG. **1**.

In the illustrated embodiment, the method **200**, shown in FIG. **2**, is employed in the context of visually programming a workflow to accomplish a given task. For present purposes, the term "programming" shall refer to the high level user function of creating a script defining, or scripting, a workflow. One embodiment of the visual programming will be disclosed more fully below. However, in general, the visual programming is performed by sequentially selecting candidate actions presented to a user to define a workflow. The list of candidate actions can be quite large, particularly on a powerful and/or robust platform. Furthermore, some candidate actions might, if selected first, produce a runtime error. In this environment, the present invention is employed filter the candidate actions by their relevance to the next selection in order to mitigate the likelihood of these kinds of adverse consequences.

Referring now to FIG. **2**, the illustrated embodiment includes a method **200**, comprising:

determining (at **205**) a context and a data requirement for a candidate action to be selected through the user interface **118**, the selection specifying an action in a workflow; and

filtering (at **210**) the candidate actions for relevance in light of the context and the data requirement.

Note that the method **200** is, in the illustrated embodiment, implemented on the computing apparatus **100** of FIG. **1**. More particularly, the processor **104** invokes the application **125** resident on the storage **106** responsive to input from the user. The application **125** then implements the method **200** with interaction from the user through the user interface **118**, as will be disclosed more fully below.

To further an understanding of the present invention, one particular implementation will discussed relative to FIG. **3**A-FIG. **3**D. Referring now to FIG. **2** and to FIG. **3**A-FIG. **3**D, the first step is to determine (at **205**) context and a data requirement for a candidate action. For instance, if the user is faced with an empty workspace **302** (i.e., no action has yet been selected), shown in FIG. **3**A, candidate actions **306** requiring some input to operate on will be less relevant since there is no input for them. For instance, a user typically would not want to select an action that rotates an image if no image is present to be rotated. In this context, the data requirement is a null. Thus, in FIG. **3**A, the list **304** of candidate actions presented to the user comprises candidate actions whose function is to retrieve, or "get," data.

If some action has previously been selected (i.e., the creation of the workflow has already begun), then the data requirements of the workflow and the candidate actions are determined. For instance, if a previous action has retrieved an image, then a candidate action that manipulates an image

would be relevant for consideration as the next selected action. In this case the data requirement is for an image. Consider, for instance, the scenario in FIG. **3**B, in which a workflow **306** is being created. An action **308** to get items from an address book has previously been selected. The output of the action **308** is of a data-type called "Address Book people." Thus, among the candidate actions **306** presented to the user in FIG. **3**B, only the two that are relevant based on context and data type are the candidate actions **310**, **312**, as indicated by the graphical elements **314**.

Once the context and the data requirement (at **205**) have been determined, then the candidate actions that might next be selected are filtered (at **210**) for relevance in light of the context and the data requirement. The list **304** of candidate actions **306** for two different filterings in two different contexts are shown in FIG. **3**A and FIG. **3**B, as discussed above. Note that the list **304** of candidate actions **306** in FIG. **3**B includes candidate actions **306** that have been determined to irrelevant as well as those that are relevant. This is an implementation specific detail also present, but not apparent, in FIG. **3**A. In the illustrated embodiment, the result of the filtering is an ordered list, in which candidate actions **306** that are both relevant and not relevant are presented. However, in alternative embodiments, the list **304** may be unordered, and only relevant candidate actions **306** presented. In still other embodiments, the list **304** may order the candidate actions **306** by their degree of relevance. The invention admits variation in this aspect.

The filtered candidate actions are then presented, in this particular embodiment, to the candidate, ordered by relevance. As was mentioned above, the assignment of relevance can take several forms, and thus the ordering may also. For instance, in some contexts, all candidate actions may be assigned the same relevance and thus any presentation will be ordered by relevance. Or, only some of the candidate actions may be assigned any degree of relevance, and either (1) only the relevant candidate actions presented or (2) all of the candidate actions may be presented, with the relevant candidate actions higher on the list. Or, the candidate actions may be assigned degrees of relevance, with the most relevant higher on the list. Note that, where candidate actions are of equal relevance, subsidiary ordering (e.g., alphabetical, last used) may be employed.

Note that the method **200**, shown in FIG. **2**, may be iterated, i.e., employed at each step in the creation of the workflow. FIG. **3**C and FIG. **3**D illustrate such iteration as two more actions **316**, **318**, respectively, are added to the workflow. The action **316**, first shown in FIG. **3**C, is an "AppleWorks Mail Merge" action that both receives and outputs "Address Book Person" data. Note that "AppleWorks Mail Merge" was one of the relevant candidate actions **314** in FIG. **3**B. Note also that, in FIG. **3**C, since the action **314** outputs the same data type as it receives in input, the same two candidate actions **306** are marked as relevant candidate actions **314** for the next selection. In FIG. **3**D, the last selected action **318** is a "Group Mailer" action, which outputs a "mail message" data type. Thus, the relevant candidate actions **314** in FIG. **3**D differ from those in FIG. **3**C. Note that the relevant candidate action **314** called "Send Outgoing Message(s)" could be selected to end the workflow in FIG. **3**D.

As was mentioned above, the illustrated embodiment is employed in a workflow scripting environment. The practice of the present invention, in one particular embodiment, shall now be discussed in the context of an exemplary workflow scripting environment. FIG. **4** conceptually illustrates, in a partial block diagram, selected portions of the hardware and software architectures, respectively, of one particular implementation of the computing apparatus of FIG. **1** for use in this workflow scripting environment. FIG. **5** illustrates one particular embodiment for scripting a workflow in accordance with one particular embodiment of the present invention.

Referring now to both FIG. **4** and FIG. **5**, the method **500** begins with the application **125** receiving (at **505**) a plurality of user inputs through the user interface **118***a*. Since the user interface **118***a* is, in the illustrated embodiment, a GUI, each user input includes selecting a graphical element (not shown) specifying an action in a workflow. However, other types of user input techniques may be employed. For instance, an input may be typed by the user in a common gateway interface ("CGI") box in a GUI, or in a command entry line in a non-graphical user interface.

Note that a "user input" may actually constitute a set of user interface actions. For instance, a "user input" may include clicking and dragging a graphical element in a GUI using the mouse **114** coupled with a keystroke on the keyboard **112**. Thus, in the illustrated embodiment, the user input may be a single click or drag using the mouse **114** or keystroke on the keyboard **112**; a set of clicks and/or drags using the mouse **114** or set of keystrokes on the keyboard **112**; or some combination of clicks and/or drags using the mouse **114** and keystrokes on the keyboard **112**.

In general, the workflow may be considered to be a number of actions intended to produce some desired result. An action may be any programmed functionality of the computing apparatus **100** that produces a user level result. The number of actions in any given workflow will typically be in the range of 5-7, but the number of actions is not material to the practice of the invention. The user will generally employ commonly known and used GUI techniques, such as "point and click" or "click and drag," with the mouse **114** to select a graphical element. The graphical element will typically be selected from among several choices presented by the application **125** through the GUI **118***a*. The choices may be presented in classic GUI fashion, e.g., in drop-down menus, pop-up windows, or sidebar panels.

The application **125** then automatically generates (at **510**, in FIG. **5**) a background script **127**, shown in FIG. **4**, for executing the actions in the workflow responsive to the user inputs. In this context, the application **125** automatically generates the script **127**. The script **127** is essentially a batch file containing a plurality of commands or pointers for performing each of the actions in the workflow. Each command includes instruction and information for implementing the specified action. The script **127** may be generated (at **510**) by the application **125** as the actions are input by the user or at some other convenient time. One other convenient time would be when the user indicates that the workflow has been fully defined, e.g., by entering a command to save the session with the application **125** or to invoke the script **127** to iterate the workflow.

The script **127** will typically be saved upon direction of the user. The user can then later invoke the script **127** from within the operating system **102** through the user interface **118***a*. Since the user interface **118***a* is GUI, the invocation may be in classic GUI fashion, e.g., through drop-down menus, pop-up windows, or sidebar panels. On invocation, the script **127** will execute the commands it contains, which will execute the actions previously specified by the user, as described above, to implement the workflow. Thus, the user will be saved the time-consuming process of performing each of the tasks separately by simply entering the one command invoking the script **127**. Note, however, that is not necessary to store the script **127**. The script **127** may be invoked prior to being stored, or subsequent to being stored in various embodiments.

Furthermore, although the invention contemplates that the script **127** will typically be invoked multiple times following its generation, such is not required by the invention.

To more fully illustrate this particular embodiment, one particular implementation of the embodiment in FIG. **4**-FIG. **5** will now be disclosed. Consider a scenario in which a user frequently culls the files **405**, including the files 405_0-405_x in the storage **106***a*, on a periodic, but routine basis. In conventional practice, a user will have to go through several steps to locate and delete the files **405** of interest. If there are numerous files **405**, this may not only be tedious in its repetition, but also time-consuming. Hence, the use of the present invention.

FIG. **6** illustrates a portion of an exemplary screen display **600** on the monitor **110** using the GUI **108***a* for the computing apparatus **400**. The screen display **600** is of an empty desktop **602**, and includes a taskbar **603**, including several icons **606** through which various applications can be invoked. In the illustrated embodiment, the application **125***a* implementing the present invention is invoked off a cascading pull-down menu **609** from the taskbar **603**. Furthermore, in the illustrated embodiment, the application **125***a* may be invoked from within a current application by clicking on the "Create Workflow Using Current Selection" selection or from the desktop by clicking on the "Create Workflow Using Selected Items" selection. However, in alternative embodiments, the application **125***a* may be invoked by clicking on one of the icons **606**, provided that one of the icons **606** is a graphical element representing the application **125***a*.

On invocation, the application **125***a* opens a new window **700**, shown in FIG. **7**. The window **700**, in the illustrated embodiment, comprises several panes **702**-**706**. The pane **702** is a header pane presenting a number of more general functions. The panes **703**-**705** are sidebar panels providing information and options to facilitate creation of the workflow. More particularly, the pane **703** presents several graphical elements Cat_0-Cat_7 identifying categories of actions from which the user may choose. Categories could be, for instance, groups of actions related to finding files, finding contacts, sending e-mails, opening text editors, etc. The pane **704** presents the user a list of candidate actions that might be selected as a portion of the workflow to be created. Note that each entry in the list of the pane **704** is a graphical element. Note also that, in some embodiments, the list of candidate actions can be defined by the selection of a category from the pane **703**. The pane **705** presents explanatory information regarding a selected or candidate action. Finally, the pane **706** provides a workspace for defining the workflow.

Note that the number, structure, presentation, and orientation of the panes **702**-**706** is implementation specific. For instance, the pane **703** of categories may be omitted, and the user may simply be presented a list of candidate actions such as those in the pane **704**. In some embodiments the information may be presented in, for instance, pop-up windows. As an example, upon invocation, the categories represented by the graphical elements Cat_0-Cat_7 may be presented in a pop-up window. After the user selects a category, the first pop-up window can be replaced by a second pop-up window presenting a list of candidate actions from which to choose. Thus, the structure and presentation of the window **700** will be implementation specific.

The user may begin creating or defining a workflow in the workspace of the pane **706** by selecting actions from the list presented in the pane **704**. The selections may be made in conventional GUI fashion. For instance, the user may click and drag an action listed in the pane **704** into the pane **706**. Alternatively, the user may double click on the action in the pane **704** and a corresponding graphical element may open in the pane **706**. Each of the user selections constitutes a user input that is received by the application **125** (at **505**, FIG. **5**).

FIG. **8** illustrates the window **700** after a workflow **800** has been created in the pane **706**. In the workflow **800**, the user has defined four actions, namely:

- an action **802** that "gets", or retrieves, selected files in the OS using an application called "Finder";
- an action **803** that filters out the retrieved files named X38B;
- an action **804** that copies the filtered files to a new location; and
- an action **805** that trashes the filtered files.

The workflow **800** can then be invoked or saved by clicking on the "Run" or "Stop" buttons **809**, **812**.

Note that the design of the interface within the pane **706** will be implementation specific. In the actions **802**-**804**, the dialog boxes associated with each action **802**-**804** include features that permit the user flexibility is defining the action. For instance, the action **802** includes a list **815** that may be scrolled through to select an application to perform the "get". Similarly, the action **802** includes a box **818** into which the user may directly type information. Other, well known GUI techniques, such as pop-up windows, may be employed.

As each of the actions **802**-**805** is selected and the associated information is entered, the application **125***a* generates the script **127***a*, both shown in FIG. **4**. In the illustrated embodiment, the script **127***a* is written in extensible markup language ("XML"), but any suitable programming language may be used. Each action will be coded into the script **127***a* with an instruction to perform the selected action and the information used to implement the instruction. However, the application **125***a* generates this code in the background, and it is not presented to the user. The workflow is instead presented to the user visually, through concrete interface blocks. The user is not required to program the tool itself, but only to arrange the building blocks (i.e., the actions **802**-**805**) and set the options and/or parameters unique to each block. The application **127***a* then uses these content-agnostic actions **802**-**805** and the connections between them as the programming tool. Unlike the prior art, there is only one conceptual unit—the action. Everything else, including the types of information passed between the actions **802**-**805** is hidden from the user. Furthermore, in the illustrated embodiment, each of the actions **602**-**605** is modular and self contained.

Referring now to FIG. **4** and FIG. **8**, at runtime, the user invokes the script **127***a*, which performs the scripted actions **802**-**805**. Assume for purposes of illustration that the files **405** comprise the files to be "gotten." Upon invocation, the script **127***a* will invoke the "Finder" application to "get" the files **405**, filter out the files 405_0-405_x that are named X38B, copy them, and discard them. Thus, instead of separately performing four OS-level commands and entering the relevant information, the user need only invoke the script **127***a*. Furthermore, the user need not be conversant with conventional programming techniques, such as use of syntax, etc., to program the script **127***a* since the application **125** handles all that in the background.

The illustrated embodiment also includes a feature in which the application **125***a* automatically determines an execution or program flow for the entered actions at runtime. The script **127***a* does not necessarily have to perform the scripted actions in the order specified in creating the workflow. In some embodiments, the order in which the actions **802**-**805** are to be performed, i.e., the flow of the program or data, can be defined by the order in which the actions **802**-**805** are selected by the user. However, in this particular embodi-

ment, the order of selection during creation is a starting point from which the application **125***a* determines the flow automatically.

Consider the scenario **900**, shown in FIG. **9**, in which the actions **901-903** are depicted. Assume that the user has selected the actions **901-903**: an action **901** to retrieve images, an action **902** to rotate images 90°, and an action **903** to copy files from one location to another. Assume the actions **901-903** were selected by the user in the create of the workflow in the order **901**, **902**, **903**. Note the directed relationship **905** shown between the actions **902**, **903** and the directed relationship **908** shown between the actions **901**, **903**. The directed relationships **905**, **908** are defined by the user in the creation of the workflow by the order in which the actions **901-903** are selected. Note that no relationship is specified between the actions **901**, **902**.

The application **125***a*, in the illustrated embodiment, starts with the first selected action, the action **901**, at runtime. In theory, however, the application **125***a* may choose any selected action **901-903** at random. If the chosen action has no parameter information missing, it is run. If parameter information is missing, then the opportunity to run is passed to the action that is to provide the information. This process iterates until an action is located that requires no input and only provides output or parameters to some other action. Once that action is located, the workflow as created is followed downward, executing each action until an action is reached that takes only input and produces no output. At this point, the workflow is considered finished and that thread of the script **127***a* is terminated.

For instance, referring now to FIG. **9**, suppose the application **125***a* selects the action **902** as a starting point. If so, the script **127***a* will be unable to execute the action **902** because it lacks the information to do so—namely, which image(s) to rotate. Thus, the application turns to the preceding selected action, the action **903**, following the directed relationship **908** back to the action **903**. The action **903**, to copy files from one location to another, takes files to copied as input and produces the copied files as output. Since the action **903** needs input, the script **127***a* follows the directed relationship **905** back up to the action **901**. The action **901**—to "get" files—requires no input and produces as output the retrieved files. Thus, the script **127***a* can execute the action **901** and follow the workflow from the action **901**, to the action **903**, and to the action **902**. If the action **902** is the last action in the script **127***a*, the script **127***a* will terminate. If the action **902** is not the last, then the script **127***a* will continue. Note that this technique can be adapted for the script **127***a* to execute multiple threads of the script **127***a* in parallel.

Thus, some alternative embodiments include a method **1000**, shown in FIG. **10**, which generally comprises:

receiving (at **1005**) a plurality of user inputs specifying a plurality of actions in a workflow; and

scripting (at **1007**) the workflow; and

automatically determining (at **1010**) an execution flow through the scripted workflow at runtime predicated on the passing of parameters between the scripted actions.

Note that these embodiments contemplate some variation in the implementation of the method **1000**.

However, as mentioned above, the illustrated embodiment, defines the execution flow at runtime as the flow defined by the user inherent in the selection of the actions. Thus, should a user select an action prior to selecting an action that would provide the input, a runtime error will occur. The application **125***a* can detect this problem during the user's selection of the actions. In this scenario, the application **125***a* presents a warning to the user notifying them of the error and provides the user with an opportunity to correct the error. The user can "click and drag" the offending action to rearrange the order of the selected actions, for instance. Alternatively, in some embodiments, the user can be presented with the information concerning the runtime error at runtime when the error occurs, and the user can go back in and correct the workflow.

As was described above, the selection of each action **802-805** is preceded by a filtering of the available candidate actions to find those that are relevant for the next selection in the workflow **800**. As is apparent from the discussion above, one consideration in this filtering is data type compatibility. For instance, consider a scenario in which a user first selects an action (not shown) that retrieves address book entries, such as that described in FIG. **3**A-FIG. **3**B. A candidate action to rotate an image would at this point would be inappropriate, since the candidate action needs an image input rather than an address book input. To ease data type compatibility concerns, the illustrated embodiment includes a technique for automatic data conversion that is transparent to the user. The present invention employs this automatic data conversion to assign degrees of relevance to candidate actions by which the list of candidate actions may be ordered.

The automatic data conversion provides data output by one specific action in a specific format that data can then be provided to another action as input in a different format completely without the user's knowledge. The conversion is also performed with no significant data loss, i.e., no loss so great the following actions or actions cannot be implemented. The automatic conversion is performed by performing other actions, in the background, that perform the conversion. Any number of these actions can be undertaken in this manner to perform the desired conversion.

Consider the workflow **1100**, shown in FIG. **11**, in which two actions **1105**, **1110** have been selected. In the workflow **1100**, the action **1105** retrieves images from the iPhoto application and the action **1110** copies them to a location specified by the user. More particularly, the user will be prompted to enter a location to which to copy the images. The action **1105** outputs data in the image format of iPhoto photographs and the action **1110** is expecting data input in the format of Finder files. The application **125***a* in the illustrated embodiment senses this mismatch and scripts the automatic data conversion as the workflow **1100** is created. If the application **125***a* cannot find a way to complete the conversion, the caller of the data conversion is notified. The notification may take place during creation of the workflow **1100** and relayed to the user, or may take place at runtime, depending on the implementation.

More particularly, on detecting the mismatch, the application **125***a* will search for a conversion action (or actions, if needed) that can convert iPhoto photographs to Finder files in one or several steps. The application **125***a* then automatically scripts those actions in the background, i.e., without the user's knowledge, into the script **127***a*. The path of data conversion is determined from a conventional, breadth-first search technique implemented by a data conversion engine **410** embedded in the script **127***a*. Suppose the OS **102**, shown in FIG. **4**, includes at least four data types. For present purposes, those data types will be identified as A-D, with A being the output of the action **1105** (i.e., iPhoto format) and D being the input to the action **1110** (i.e., Finder file format). The data conversion engine **410** will first search for a conversion functionality that converts from A to D. The data conversion engine **410** then searches for conversion functionalities that convert to anything from A and anything to D, and moves through this process for B and C. The data conversion engine

410, through this breadth-first search technique produces a list of the identified conversion functionalities from which the application **125***a* then selects the actions that are then scripted into the script **127***a*.

In the illustrated embodiment, the data conversion engine **410** uses a variant Universal Type Interface ("UTI"), or type interface, to specify the input and output type from individual actions, or conversion functionalities. Examples include, for instance, com.apple.iphoto.photo and public.alias. More particularly, the data conversion engine **410** employs a reverse-Domain Name Service ("DNS") naming style for specification of data type inputs and outputs. This type interface builds on the reverse-DNS naming convention used by Mac OS X. The type identification formalizes this naming convention for broader use in Mac OS X.

The syntax of a uniform type identifier matches the Internet's Domain Name syntax as specified in IETF RFC-1035, with three exceptions: (1) every identifier is required to be fully specified (i.e., have at least one domain label preceding the leaf label), (2) the label order is the reverse of that used for DNS names, and (3) in addition to the strict DNS character set (A-Z, a-z, 0-9, "-" and "."), Unicode graphic characters greater than U+007F (non ASCII) are allowed. As with DNS names, both upper- and lower-case letters are permitted in uniform type identifiers, but identifier comparisons are not case-sensitive.

Thus, the illustrated embodiment employs a method **1200**, illustrated in FIG. **12**, automatically, and seamlessly, converts disparate data types in a workflow, including:

- receiving (at **1205**) a plurality of user inputs, each user input specifying an action in a workflow;
- identifying (at **1210**) a data type incompatibility between two of the user inputs; and
- generating (at **1215**) a background script for executing the actions in the workflow responsive to the user inputs, the background script including code for creating a background action for converting one of the two identified, incompatible data types to the second.

Using these types of techniques, the data conversion engine **410** identifies the data types output by the various actions, including the action **1105**, **1110** selected in the workflow **1100**. The data conversion engine **410** can then determine whether there are converters available, chain them together appropriately, and the call them in order to do each transformation. The length of this transformation chain is unbounded by the data conversion engine **410** in the illustrated embodiment.

Note, however, that not all embodiments need to automatically and seamlessly convert data. In alternative embodiments, subsequent lists of candidate actions from which to choose may be restricted for data type compatibility. Or, the user may be permitted to select actions that will expressly perform the data type conversions. The automatic, seamless conversion of data types, however, provides flexibility and eases the user's burden—in some respects—in creating the workflow.

The automatic, seamless data type conversion does, however, generally tend to increase the number of candidate actions that can be presented to the user in the pane **704** of the window **700**, first shown in FIG. **7**. Consequently, the filtering process of the illustrated embodiment employs data type conversion as a measure for determining the relevance of a given action as a candidate for selection in the workflow. Since the relevance is defined relative to what has gone before in the creation of the workflow, it depends in some degree on the context. For instance, referring to FIG. **11**, the actions **1105**, **1110** each output data in a different type. Thus, the relevance will depend in part on whether any actions have been selected and, if so, whether the last selected action was the action **1105** or the action **1110**.

As is shown in FIG. **13**A, the application **125***a* first determines (at **1305**) an action on which to base the relevance ranking. If there are no selected actions (at **1310**), then the relevance of all candidate actions may be, for instance, set to the same level. Or, candidate actions that accept any data input may be accorded the highest relevance and all others accorded the lowest relevance. In the illustrated embodiment, the user may click on a previously selected action to identify it as the one by which to determine the relevance of the candidate actions. If the user fails to identify a particular action, then the application **125***a* may identify the last previously selected action.

More particularly, if there are no previously selected actions for the workflow, then candidate actions are accorded relevance by the range of data types that they will accept as input. For instance, actions that will receive anything as input are accorded the highest relevance, followed by actions that will receive a threshold number of data types, followed by actions that meet a second (lesser) threshold, followed by actions that do not receive input. In general, though, the application **125***a* presents (at **1315**) to the user a list of candidate actions suitable for use as workflow starting points.

If some actions have already been selected (at **1310**), then relevance is accorded by the degree to which data type conversion will be required (at **1320**). Candidate actions requiring no conversion, i.e., those that can accept the output of the identified action directly as input, are accorded the highest relevance. Candidate actions that will need a threshold number of data type conversions receive the next highest relevance followed by candidate actions that meet a second (higher) threshold number of data type conversions. Candidate actions whose input data type is one to which the output data type of the last-selected action cannot be converted are accorded the lowest relevance.

Thus, in the illustrated embodiment, the user opens the workflow scripting window **700**, shown in FIG. **7**, by invoking the application **125***a*, shown in FIG. **4**, from the pull-down menu **609**, shown in FIG. **6**. When the window **700** opens, the application **125***a* filters the candidate actions with which the workflow may be started using the method **1200**, shown in FIG. **12**. The user then begins creating a workflow, e.g., the workflow **800**, shown in FIG. **8**. As the workflow is created, the application **125** looks for data-type mismatches and seamlessly and automatically scripts conversions to cure them in accordance with the method **1200**, shown in FIG. **12**. The application **125** also, after each action in the workflow is selected, again filters the list of candidate actions. When the user is finally finished creating the workflow, they save the script **127***a*, also shown in FIG. **4**, through the application **125***a*.

The user can then subsequently invoke the script **127***a* through the GUI **118***a* of the operating system **102** to execute the defined workflow. As the script **127***a* is executed, the script **127***a* employs the scripted data-type conversions using the data conversion engine **410** and automatically determines a path through the defined workflow in accordance with the method **1000**, shown in FIG. **10**. The script **127***a* can also be invoked repeatedly to save the user the time and difficulties of performing each action in the workflow manually each time the user wishes to perform the task.

The present invention therefore provides a workflow/scripting environment in which each self-contained action is backed by a script of information, but is visually presented to

the user as a concrete interface block. As noted above, the user is not required to program the tool itself, only to arrange the actions and set the options and/or parameter unique to each action. The actions are then evaluated by criteria inherent in the built-in engine and executed accordingly.

Ultimately, the user does not "program" the computing apparatus in the conventional sense, or even in the sense of what are known as conventional "visual programming" techniques. The user instead performs the actions necessary to execute the desired task one time, and the application generates an automatic script to actually program the computing apparatus. Thus, the programming knowledge required in prior art "visual programming" techniques is not a requisite for use of the present invention. In the illustrated embodiment, the user need not even worry about data-type mismatches or errors in the order of the workflow. The user performs typical GUI functions in the workflow scripting environment, which then automatically, and in the background, performs the associated programming tasks.

Note, however, that the present invention is not limited the determination of relevance as described relative to FIG. **4**-FIG. **12**. For instance, an alternative embodiment might assign relevance differently than is shown in FIG. **13**A. One such alternative is illustrated in FIG. **13**B, with like parts bearing like numbers. In this embodiment, the application **125***a* essentially searches (at **1325**) the candidate actions for those that accept as input the data type output by the last selection action and marks (at **1330**) them as relevant. In this particular embodiment, relevance is a binary function—either the candidate action is relevant or it is not.

In this particular embodiment, the relevance of each candidate action can be reassessed for every iteration of the method or it can be carried over. For instance, when a candidate action is selected, the relevance of each candidate action can be reset so that relevance is determined relative only to the immediately preceding, selected action. On the other hand, once a candidate action has been identified as relevant to a selected action, it may carry that relevance until the workflow is fully defined. The relevant candidate actions presented to the user will then reflect their relevance at any point in the workflow creation. This may useful where, for instance, the user may wish to select an action to manipulate the output of an earlier selected action from two or three actions up the workflow. This approach may be extended by incrementing each candidate action each time it is relevant so that candidate actions relevant to more selected actions are ranked as more relevant than those relevant to fewer, or no selected actions. Thus the present invention admits wide variation in the determination of relevance in the present invention.

The illustrated embodiment also has several other notable features. For instance, the invention implements an extensible user interface, and does so with a parametric script. The user interface is extensible because actions may be imported into the environment from other parties (internal or external) and used to create the parametric scripts generated by the application. When invoked, the user may then interface with the extensible actions from another party, and thus the application implements an extensible user interface. The script is parametric in that the user specifies the parameters for the script during the creation of the workflow. Furthermore, because the specified actions are represented graphically, the workflow is reconfigurable by "clicking and dragging" specified actions into an order different than what was originally specified. Note how the module nature of the action facilitates the reconfiguration of the workflow.

This concludes the detailed description. The particular embodiments disclosed above are illustrative only, as the invention may be modified and practiced in different but equivalent manners apparent to those skilled in the art having the benefit of the teachings herein. Furthermore, no limitations are intended to the details of construction or design herein shown, other than as described in the claims below. It is therefore evident that the particular embodiments disclosed above may be altered or modified and all such variations are considered within the scope and spirit of the invention. Accordingly, the protection sought herein is as set forth in the claims below.

What is claimed is:

1. A computer-implemented method for use in a computing apparatus, the method comprising:

automatically determining a context and a data requirement for a candidate action to be selected, the selection specifying an action in a workflow; and

automatically filtering the candidate actions for relevance in light of the context and the data requirement, including one of:

according a relevance to each of the candidate actions relative to the degree to which data type conversion will be required between the selected action and the respective candidate action; and

according a relevance to each of the candidate actions that require no data conversion to accept the output of the previously selected action as input.

2. The computer-implemented method of claim **1**, further comprising presenting the filtered candidates to the user.

3. The computer-implemented method of claim **2**, further comprising presenting the filtered candidates through a graphical user interface.

4. The computer-implemented method of claim **1**, further comprising ordering the filtered candidate actions by their relevance.

5. The computer-implemented method of claim **1**, wherein determining the context and the data requirement includes determining that no candidate action has previously been selected.

6. The computer-implemented method of claim **5**, wherein the determining the data requirement includes determining that the data requirement is a null.

7. The computer-implemented method of claim **5**, wherein filtering the candidate actions for relevance includes one of:

according all candidate actions the same relevance;

according candidate actions that accept any data input the highest relevance and all others the lowest relevance; and

according by the range of data types that the candidate actions will accept as input.

8. The computer-implemented method of claim **1**, wherein determining the context and the data requirement includes identifying a previously selected action.

9. The computer-implemented method of claim **8**, wherein determining the data-type includes determining the data-type of the output of the identified, previously selected action.

10. The computer-implemented method of claim **8**, wherein filtering the candidate actions for relevance includes according a relevance to the respective candidate actions as a function of data compatibility between the previously selected action and the respective candidate action.

11. The computer-implemented method of claim **8**, wherein filtering the candidate actions for relevance includes according a relevance as a function of the number of previously selected actions to which the candidate action is relevant.

12. The computer implemented method of claim 1, further comprising graphically presenting a workflow into which the candidate actions may be scripted.

13. The computer implemented method of claim 12, further comprising automatically converting data types between selected actions in the scripted workflow.

14. The computer implemented method of claim 12, further comprising automatically determining an execution flow through the workflow at runtime.

15. A computer-readable memory encoded with instructions that, when executed by a computing device, perform a method for use in a computing apparatus, the method comprising:

automatically determining a context and a data requirement for a candidate action to be selected, the selection specifying an action in a workflow; and

automatically filtering the candidate actions for relevance in light of the context and the data requirement, including one of:

according a relevance to each of the candidate actions relative to the degree to which data type conversion will be required between the selected action and the respective candidate action; and

according a relevance to each of the candidate actions that require no data conversion to accept the output of the previously selected action as input.

16. The computer-readable memory of claim 15, wherein the encoded method further comprises presenting the filtered candidates to the user.

17. The computer-readable memory of claim 15, wherein the encoded method further comprises ordering the filtered candidate actions by their relevance.

18. The computer-readable memory of claim 15, wherein determining the context and the data requirement in the encoded method includes determining that no candidate action has previously been selected.

19. The computer-readable memory of claim 15, wherein determining the context and the data requirement in the encoded method includes identifying a previously selected action.

20. A computing apparatus, comprising:

a computing device;

a computer-readable memory communicating with the computing device; and

an application residing in the computer-readable memory that, when invoked, automatically:

determines a context and a data requirement for a candidate action to be selected, the selection specifying an action in a workflow; and

filters the candidate actions for relevance in light of the context and the data requirement, including one of:

according a relevance to each of the candidate actions relative to the degree to which data type conversion will be required between the selected action and the respective candidate action; and

according a relevance to each of the candidate actions that require no data conversion to accept the output of the previously selected action as input.

21. The computing apparatus of claim 20, wherein the programmed method further comprises presenting the filtered candidates to the user.

22. The computing apparatus of claim 20, wherein the programmed method further comprises ordering the filtered candidate actions by their relevance.

23. The computing apparatus of claim 20, wherein determining the context and the data requirement includes determining that no candidate action has previously been selected.

24. The computer implemented method of claim 20, wherein the application is further capable of graphically presenting a workflow into which the candidate actions may be scripted.

25. A computer-implemented method for scripting a workflow, comprising:

generating a user interface having one or more panes;

displaying in a first one of said panes a user selectable list of scriptable candidate actions;

displaying a graphical depiction of a workflow in progress, including at least one action previously selected from said list of scriptable candidate actions; and

automatically filtering the user selectable list of scriptable candidate actions including one of:

according a relevance to each of the candidate actions relative to the degree to which data type conversion will be required between the selected action and the respective candidate action; and

according a relevance to each of the candidate actions that require no data conversion to accept the output of the previously selected action as input.

26. The computer-implemented method of claim 25, wherein:

generating the user interface includes generating a graphical user interface presenting a plurality of panes;

displaying the user selectable candidate actions includes displaying the candidate actions in a first one of the panes; and

displaying the graphical depiction includes displaying the graphical depiction in a second one of the panes.

27. The computer-implemented method of claim 25, wherein generating the user interface includes generating an extensible user interface.

28. The computer-implemented method of claim 25, wherein filtering the user selectable list of scriptable candidate actions includes:

determining a context and a data requirement for a candidate action to be selected, the selection specifying an action in a workflow; and

filtering the candidate actions for relevance in light of the context and the data requirement.

29. The computer-implemented method of claim 25, further comprising ordering the filtered candidate actions by their relevance.

30. A computer-readable memory encoded with instructions that, when executed by a computing device, performs a method for scripting a workflow, the method comprising:

generating a user interface having one or more panes;

displaying in a first one of said panes a user selectable list of scriptable candidate actions;

displaying a graphical depiction of a workflow in progress, including at least one action previously selected from said list of scriptable candidate actions; and

filtering the user selectable list of scriptable candidate actions, including one of:

according a relevance to each of the candidate actions relative to the degree to which data type conversion will be required between the selected action and the respective candidate action; and

according a relevance to each of the candidate actions that require no data conversion to accept the output of the previously selected action as input.

31. The computer-readable memory of claim 30, wherein, in the encoded method:

generating the user interface includes generating a graphical user interface presenting a plurality of panes;

displaying the user selectable candidate actions includes displaying the candidate actions in a first one of the panes; and

displaying the graphical depiction includes displaying the graphical depiction in a second one of the panes.

32. The computer-readable memory of claim **30**, wherein generating the user interface in the encoded method includes generating an extensible user interface.

33. The computer-readable memory of claim **30**, wherein filtering the user selectable list of scriptable candidate actions in the encoded method includes:

determining a context and a data requirement for a candidate action to be selected, the selection specifying an action in a workflow; and

filtering the candidate actions for relevance in light of the context and the data requirement.

34. The computer-readable memory of claim **30**, wherein the encoded method further comprises ordering the filtered candidate actions by their relevance.

35. A computer-implemented method for scripting a workflow, comprising:

receiving a plurality of user selections of actions to be included within a scriptable workflow;

filtering a list of candidate actions for selection into the scriptable workflow, including one of:

according a relevance to each of the candidate actions relative to the decree to which data type conversion will be required between the selected action and the respective candidate action; and

according a relevance to each of the candidate actions that require no data conversion to accept the output of the previously selected action as input; and

presenting the filtered list to a user.

36. The computer-implemented method of claim **35**, wherein filtering the list of candidate actions includes:

determining a context and a data requirement for a candidate action to be selected, the selection specifying an action in a workflow; and

filtering the candidate actions for relevance in light of the context and the data requirement.

37. The computer-implemented method of claim **35**, further comprising ordering the filtered candidate actions by their relevance.

38. A computing apparatus, comprising:

a computing device;

a computer-readable memory communicating with the computing device; and

an application residing in the computer-readable memory that, when invoked, automatically:

receives a plurality of user selections of actions to be included within a scriptable workflow;

filters a list of candidate actions for selection into the scriptable workflow, including one of:

according a relevance to each of the candidate actions relative to the degree to which data type conversion will be required between the selected action and the respective candidate action; and

according a relevance to each of the candidate actions that require no data conversion to accept the output of the previously selected action as input; and

presents the filtered list to a user.

39. The computing apparatus of claim **38**, wherein filtering the list of candidate actions includes:

determining a context and a data requirement for a candidate action to be selected, the selection specifying an action in a workflow; and

filtering the candidate actions for relevance in light of the context and the data requirement.

40. The computing apparatus of claim **38**, wherein the application is further capable of ordering the filtered candidate actions by their relevance.

* * * * *

UNITED STATES PATENT AND TRADEMARK OFFICE

CERTIFICATE OF CORRECTION

PATENT NO. : 7,428,535 B1
APPLICATION NO. : 10/877306
DATED : September 23, 2008
INVENTOR(S) : Eric S. Peyton et al.

It is certified that error appears in the above-identified patent and that said Letters Patent is hereby corrected as shown below:

In column 19, line 29, delete “decree” and insert -- degree --.

Signed and Sealed this

Eighteenth Day of November, 2008

JON W. DUDAS
Director of the United States Patent and Trademark Office